Домашнее задание
Неделя 5. API и внешние источники

Бизнес-кейс: заказы внешней платформы

Интернет-магазин «СеверМаркет» получает заказы через систему NorthCart. Аналитикам нужны заказы и их позиции в PostgreSQL. Сейчас сотрудники выгружают только первую страницу API, а повторный запуск создаёт дубликаты. Изменения статусов и состава заказа попадают в отчёт нерегулярно.

Ваша задача - написать загрузчик текущего состояния заказов в staging: получить весь выбранный набор, проверить записи, сохранить корректные данные и подтвердить результат. Частичную выгрузку нельзя объявлять успешной. Все компании и данные вымышлены.

8 заданий составляют один загрузчик. В архиве `DE_Week5_Practice.zip` находятся готовый сервер, два снимка данных, `setup.sql`, `API.md` и инструкция запуска `README.md`. Сервер писать не нужно. Для выполнения используйте Python, Requests, Psycopg 3 и учебную PostgreSQL.

1. Подключитесь к источнику и проверьте контракт

Команда интеграций предоставила доступ к аккаунту `shop_de`. Сначала нужно убедиться, что запрос выбирает нужные данные и не скрывает ошибки доступа.

1. Запустите сервер по README. Выполните GET к `http://127.0.0.1:8765/v1/orders`. Токен получите из `DE5_API_TOKEN` и передайте в заголовке Bearer; параметры передайте через `params`. Задайте явный `timeout`.

2. Передайте `account_id=shop_de`, `snapshot=initial`, `scenario=normal`, `limit=4`. Период: от `2026-09-24T00:00:00Z` включительно до `2026-09-27T00:00:00Z` исключительно. Поле отбора - `updated_at`.

3. Проверьте HTTP-статус, затем разберите JSON. Найдите список заказов, курсор и сведения о размере выборки. Зафиксируйте, сколько записей пришло в первом ответе и сколько заявлено во всём наборе.

4. Отдельным пробным запросом задайте `status=paid` и убедитесь, что фильтр работает. В основной загрузке уберите его: нужны все статусы. Пробный запрос не должен менять общий прогресс загрузки.

Результат: рабочий запрос с безопасным хранением настроек и понятным назначением каждого параметра. Первый ответ основного запроса содержит 4 заказа, весь набор - 18.

2. Получите все страницы без потерь

Аналитик заметил, что в источнике заказов больше, чем в базе. Причина должна обнаруживаться по итогам загрузки, а не после публикации отчёта.

Контракт страницы

Поле	Ожидаемый формат
`data`	Обязательный массив заказов. Пустой массив допустим.
`next_cursor`	Обязательное поле: непустая строка для продолжения или null для конца.
`meta.total_count`	Обязательное неотрицательное целое количество записей всего выбранного набора, включая некорректные.
`meta.snapshot`	Имя выбранного снимка: initial или updated.

1. Разделите получение одной страницы и обход всех страниц на функции. Проверяйте JSON-объект, типы обязательных полей и наличие информации о продолжении до обработки записей.

2. Первый запрос отправьте без курсора. Дальше передавайте точное значение `next_cursor`; остальные параметры сохраняйте. Не вычисляйте курсор самостоятельно.

3. Завершайте обход только после корректного ответа с `next_cursor = null`. Ошибка запроса, отсутствие поля и достижение защитного лимита не являются концом выгрузки.

4. Добавьте проверку повторяющихся курсоров и предел 100 страниц для этой практики. Достижение предела должно давать ошибку. Запись не должна обрабатываться повторно из-за повторной HTTP-попытки.

5. Ведите отдельно число успешных страниц, HTTP-попыток и полученных заказов. Сверьте количество и набор идентификаторов; один пропуск и один повтор не должны скрыться за одинаковой общей цифрой.

Проверьте два варианта

Для `scenario=normal` ожидаются **5 успешных страниц и 18 записей**. До проверки полей все `order_id` различны: от `000001` до `000018`. Это проверка полноты получения, а не утверждение, что все записи пригодны для загрузки.

Для `scenario=empty_then_data` первая страница пуста, но содержит курсор. Ожидаются **6 успешных страниц и те же 18 записей**. Количество строк первой страницы не должно определять завершение.

Временный HTTP-сбой может увеличить число попыток, но не число обработанных страниц и записей.

3. Подготовьте заказы и позиции

Бизнесу нужны отдельные наборы заказов и товарных позиций. Одна повреждённая позиция не должна приводить к сохранению неполного заказа.

Поле источника	Правило подготовки
`order_id`	Непустая строка; ведущие нули сохраняются.
`customer`	Обязательный объект с непустым строковым id либо null. Для null сохраняйте customer_id = None.
`status`, `currency`	Статусы: pending, paid, cancelled, refunded. Валюта только EUR; значения без самовольного исправления.
`total_amount`	Неотрицательная строковая сумма с точкой и максимум двумя десятичными знаками; преобразуйте через Decimal.
`created_at`, `updated_at`	ISO 8601 с часовым поясом; время изменения не раньше создания.
`items`	Обязательный массив. Пустой допустим; null и отсутствие поля являются ошибкой.
Поля каждой позиции	Непустые строки line_id и sku; целое quantity > 0; строковое unit_price по денежным правилам. line_id уникален внутри заказа.

Точные допустимые форматы и пределы значений описаны в `API.md`. Проверьте равенство `total_amount` сумме `quantity * unit_price`. Для пустого массива сумма должна быть нулём. Это стоимость состава заказа; статус refunded не делает её отрицательной.

1. Подготовьте один словарь заказа и отдельный список его позиций. Позиции должны содержать идентификатор родительского заказа. Заказ без позиций остаётся в наборе заказов.

2. Проверяйте весь заказ до записи в базу. При ошибке поля или любой позиции отклоняйте заказ целиком. Сохраняйте исходный объект, run_id, raw_response_id, номер записи на странице и понятную причину в JSON Lines.

3. Новые неизвестные поля сохраняйте в архиве, но не переносите автоматически в целевую структуру. Обработчик не должен считать исчезновение обязательного поля допустимым расширением.

4. Ведите отдельные счётчики заказов и позиций. Не копируйте общую сумму заказа в каждую позицию для последующего суммирования.

Контроль исходной выборки: 18 получено, 14 принято, 4 отклонено; у принятых заказов 19 позиций. Сумма принятых заказов - **470.75 EUR**. Причины отклонения определите по данным и контракту.

4. Сохраните исходные ответы и staging

После сбоя поставщик может вернуть уже изменившиеся данные. Поэтому у каждой сохранённой записи должна быть связь с конкретным полученным ответом.

Архив ответов

Сохраняйте тело каждого полученного HTTP-ответа до разбора JSON, в том числе ответа с ошибкой. Используйте отдельный `raw_response_id` и отдельный каталог запуска. Для точного сохранения тела используйте байты ответа, а не повторную сериализацию разобранного JSON.

Рядом сохраняйте метаданные: run_id, raw_response_id, время получения в UTC, endpoint, безопасные параметры периода и страницы, HTTP-статус и путь к телу. Не сохраняйте Authorization и пароль базы. Если ответа не было, отразите это в журнале, не создавая фиктивные данные страницы.

Таблицы

Выполните `setup.sql` в учебной базе. Схема de5_hw используется как staging текущего состояния.

Таблица	Что представляет строка
`orders`	Текущее принятое состояние одного заказа источника и аккаунта.
`order_items`	Одна позиция текущего состояния заказа; ключ включает line_id.
`load_runs`	Одна попытка выполнения загрузчика с её статусом и счётчиками.
`load_state`	Успешно обработанная верхняя граница периода для источника и аккаунта.

1. Создайте новый UUID запуска, вставьте строку running в load_runs и зафиксируйте её отдельно. Для заказов используйте source_system = northcart и source_account_id = shop_de.

2. Откройте одну транзакцию данных на весь небольшой запуск. Передавайте SQL-значения параметрами. Заказ и его позиции должны сохраняться согласованно; для jsonb используйте адаптацию Psycopg.

3. Заполняйте технические поля orders: source_updated_at, received_at, run_id, raw_response_id, record_no и payload. Номер record_no начинается с 1 внутри страницы; при пропуске старой или одинаковой версии технические поля сохранённой строки не меняются.

4. После полного обхода и сверки зафиксируйте заказы, позиции, успешный статус запуска и completed_to одной транзакцией. При ошибке откатите её и затем отдельно установите failed. Архив и отклонения сохраняются для разбора.

Успех с отклонёнными заказами разрешён: каждая запись должна быть классифицирована, а причина каждого отклонения сохранена. Статус в таблице - succeeded; наличие отклонений видно по rejected_count.

5. Обработайте повторный запуск и изменения

На следующий день NorthCart меняет статусы, дополняет состав заказа и добавляет новые заказы. Загрузчик должен применить изменения без размножения строк и без возврата к старым состояниям.

Правило версии

Ключ заказа: `(source_system, source_account_id, order_id)`. Версию определяет `updated_at`. Для нового ключа нужна вставка; для более новой версии - обновление заказа и замена полного набора его позиций. Старую версию пропускайте.

При одинаковом времени сравнивайте подготовленные бизнес-поля и позиции по line_id. Одинаковые значения означают пропуск. Разные значения означают конфликт источника и откат запуска. Технические поля и неизвестные дополнительные поля не участвуют в этом сравнении.

Последовательность проверок

1. Выполните исходную загрузку initial за период [24 сентября, 27 сентября), затем повторите её. Во втором запуске не должно быть новых или обновлённых заказов; 14 принятых заказов должны попасть в пропущенные.

2. Переключитесь на snapshot=updated. Выгрузите период от 2026-09-26T00:00:00Z включительно до 2026-09-28T00:00:00Z исключительно. Перекрытие с предыдущим периодом намеренное; используйте тот же загрузчик.

3. Проверьте более новый статус 000003, изменившиеся статус и позиции 000008, появление 000019 и 000020. Заказ 000006 оставьте в staging: источник не передаёт удаления, а отсутствие в дельте не доказывает удаление.

4. После успешной дельты повторно получите старый снимок initial за исходный период. Заказы 000003 и 000008 не должны вернуться к старой версии. Общая граница completed_to не должна уменьшиться.

5. На исходно загруженных данных проверьте scenario=conflict_same_version. Изменённая сумма без нового updated_at должна вызвать ошибку; прежние данные и прогресс должны сохраниться.

Контроль дельты

Получено **10 заказов**, принято **6**, отклонено **4**. Среди принятых: **2 вставки, 2 обновления и 2 пропуска**. После фиксации в staging должно быть **16 заказов и 25 позиций**, сумма заказов **826.00 EUR**.

completed_to после исходной загрузки: 2026-09-27T00:00:00Z; после дельты: 2026-09-28T00:00:00Z. Обновляйте эту границу только после успешной фиксации соответствующих данных. При повторе старого периода сохраняйте прежние completed_to и его run_id.

6. Реализуйте управляемые повторы запросов

Источник иногда недоступен или ограничивает частоту запросов. Ошибку доступа нельзя исправлять бесконечными повторами, а временный сбой не должен приводить к потере страницы.

Установите **не более 3 попыток одного GET, включая первую**. Для временных ошибок соединения, тайм-аутов и 500/502/503/504 используйте паузы 1 и 2 секунды. Для 429 учитывайте Retry-After: пауза не меньше базовой и указанной сервером. В учебном API заголовок содержит целое число секунд.

Повторяйте тот же запрос с тем же курсором. Ошибки 400/401/403/404, неверный JSON и неожиданную структуру страницы автоматически не повторяйте. Ошибка PostgreSQL не должна запускать заново весь блок HTTP-запроса.

Проверка	Как воспроизвести	Ожидаемое поведение
Временный 503	transient_503	Один повтор второй страницы, затем полный результат.
Лимит запросов	rate_limit_once	Пауза не менее 1 секунды, затем повтор.
Постоянный 503	persistent_503	Три попытки второй страницы, затем ошибка.
Тайм-аут чтения	timeout_once; read timeout 0.2 с	Повторить запрос; результат совпадает с normal.
Повреждённый JSON	invalid_json	Остановка, без объявления пустого набора.
Неверный тип data	wrong_structure	Остановка из-за структуры страницы.
Нет next_cursor	missing_cursor	Ошибка контракта, не успешный конец.
Повтор курсора	cursor_loop	Остановка без зацикливания.
400 / 401 / 403 / 404	limit=6 / неверный токен / another_shop / неверный путь	Одна попытка; исправление параметров, доступа или адреса.

Используйте initial, исходный период, limit=4 и параметр scenario. Для каждого независимого теста выберите известное состояние базы. Перед повторной проверкой разового сбоя перезапустите сервер: его счётчики хранятся в памяти.

Для transient_503, rate_limit_once и timeout_once ожидаются 5 успешно полученных страниц, 6 HTTP-попыток и те же 18 записей. Для постоянного сбоя вторая страница не получена; третью запрашивать нельзя.

7. Проверьте откат и восстановление

Оператор видит, что первая страница уже обработана, но следующая недоступна. Нужно доказать, что в staging не появилось частично обновлённое состояние, а прогресс не перескочил через ошибку.

Проверка A. Сбой API при первой загрузке

1. Начните с пустых orders, order_items и load_state в отдельной учебной базе либо восстановленной схеме de5_hw. Выполните исходную загрузку с persistent_503.

2. Убедитесь, что первая страница была получена, а на второй исчерпаны три попытки. После завершения ошибки в orders и order_items должно остаться 0 строк, записи прогресса быть не должно.

3. Проверьте failed в load_runs, ненулевой код завершения процесса и сохранённый архив ответов. Не включайте откатившиеся вставки в число сохранённых строк.

4. Повторите запуск с normal и новым run_id. Результат должен совпасть с обычной исходной загрузкой: 14 заказов, 19 позиций, 470.75 EUR.

Проверка B. Сбой после записи данных

Начните с успешно загруженного исходного состояния. В загрузку дельты добавьте управляемое исключение после записи второй страницы, до COMMIT и обновления общей отметки прогресса. Такой сценарий можно включать отдельным тестовым аргументом.

После отката должны сохраниться **14 заказов, 19 позиций и 470.75 EUR**. Статусы и состав 000003 и 000008 должны остаться исходными, completed_to - 27 сентября. Затем выполните дельту без искусственного сбоя и получите контрольный результат задания 5.

Проверка C. Данные без событий и расширение ответа

В отдельном тесте запросите период [2026-10-01T00:00:00Z, 2026-10-02T00:00:00Z). Это успешный пустой набор: одна страница, 0 заказов, без ошибки разбора. Не смешивайте его отметку прогресса с основной последовательностью сентября.

Проверьте new_field на исходной выборке. Поле promotion_note должно сохраниться в архиве, а подготовленные данные и их контрольные суммы - остаться прежними. Расширение ответа не должно считаться изменением бизнес-версии заказа.

Для каждого теста зафиксируйте начальное состояние, параметры, ожидаемый результат и фактические количество строк, суммы, статус и прогресс. Проверяйте базу после завершения транзакции, а не только сообщения программы.

8. Соберите загрузчик и подтвердите результат

Другой инженер должен суметь запустить обработку и понять по журналу, что произошло. Соберите функции запроса, пагинации, проверки, преобразования, записи и сверки в один управляемый скрипт.

1. Добавьте точку входа и аргументы командной строки: адрес API, период, аккаунт, snapshot, scenario и limit. Настройки тайм-аутов должны позволять выполнить тест задания 6. Токен и параметры подключения получайте из окружения.

2. Журналируйте run_id, этап, номер страницы, номер попытки, HTTP-статус, длительность и счётчики. Не выводите токены, пароль, полные заголовки, тела ответов и полный URL с курсором. Сообщение об успешной записи выводите после COMMIT.

3. При succeeded завершайте процесс с кодом 0; при failed - с ненулевым кодом. Для успеха с отклонениями явно выводите rejected_count и путь к файлу причин.

4. В README своего загрузчика опишите рабочие команды, зависимости, переменные окружения, правила периода и версий, транзакцию, архив, отклонения и ограничения удалений. Команды проверьте фактическим запуском.

Сверка по заказам после успешной фиксации

```
получено = принято + отклонено
принято = вставлено + обновлено + пропущено
```

Запуск	Получено / принято / отклонено	Вставлено / обновлено / пропущено
Исходный	18 / 14 / 4	14 / 0 / 0
Повтор исходного	18 / 14 / 4	0 / 0 / 14
Дельта после исходного	10 / 6 / 4	2 / 2 / 2
Старый снимок после дельты	18 / 14 / 4	0 / 0 / 14

Проверьте SQL-запросами уникальность ключей, отсутствие позиций без заказа и совпадение суммы позиций с total_amount каждого заказа. Отдельно подтвердите гостевой заказ с NULL customer_id и сохранение заказов без позиций.

После исходной загрузки: **14 заказов, 19 позиций, 470.75 EUR**. После дельты: **16 заказов, 25 позиций, 826.00 EUR**. Сумма шести принятых заказов самой дельты - **481.25 EUR**; она не равна приросту суммы всей таблицы, потому что включает обновления и повторы.

Итоговая сумма служит технической сверкой стоимости состава заказов. Это не показатель выручки. Отклонённые записи и откатившиеся изменения в сохранённые суммы не включаются.